CS149 Assignment 3 复刻报告：CUDA 上的 SAXPY、并行前缀和与保序圆渲染器

ShaneLiu04*

2026 年 10 月

摘要

这份报告复刻 Stanford CS149（Fall 2024）Assignment 3「A Simple Renderer in CUDA」的三个部分。本实验目录原有的 numpy 版（统计 work/span、乱序误差和 tile 分桶的测试次数）保留下来，作为算法层面的对照。GPU 部分全部用 CUDA C++ 手写，在一块 RTX 5090 上测量（sm_120，170 个 SM，96 MB L2，GDDR7 标称 1.79 TB/s，PCIe 5.0 x16）。三个部分各回答一个问题。

第一部分问：最简单的访存受限 kernel 在 GPU 上被什么卡住？SAXPY 在 $N = 2^{28}$ 时，kernel 跑到 1578 GB/s，是标称显存带宽的 88%。可一旦把 `cudaMemcpy` 算进来，端到端只剩 53.8 GB/s（pinned 内存）和 10.5 GB/s（pageable 内存），kernel 只占总时间的 3.4%。用 4 个 stream 分块流水、让上行和下行拷贝重叠，可以到 77.1 GB/s。

第二部分问：同样是 exclusive scan，为什么不同写法会差出十几倍？六种实现在 10^3–2^{28} 的长度上（含非 2 的幂）都与 CPU 逐元素一致。2^{28} 时，starter 结构的 global-memory Blelloch 用 16.75 ms，shared-memory 块内 scan 加递归块和用 2.82 ms，`cub::DeviceScan` 用 1.41 ms。按各自实际的显存流量（约 $84N$、$16N$、$8N$ 字节）折算，三者都在约 1.52 TB/s，所以差距来自搬了多少字节，而不是算力。小数组则由 launch 次数决定：$N = 1000$ 时，naive 版 20 次 launch 用 49.6 μs，单 block 版只要 4.4 μs。基于 scan 的 `find_repeats` 在 10^8 时用 1.87 ms，比 `thrust::copy_if`（0.28 ms）慢 6.6 倍，原因同样是多遍访存。

第三部分问：渲染器怎样既并行又保住混合顺序？在 9 个 1024×1024 场景上，starter 式的「一线程一圆」kernel 除 rgb（24.3%）外，有 48.5%–100% 的像素出错，而且多数场景两次运行结果不同。四个正确版本在 90 个（场景, 阶段, tile）组合上与 CPU 参考的最大绝对误差为 0（snow 场景为 2.98×10^{-7}，来自 `expf`）。用 scan 做全局分桶、再用 shared memory 批量取圆之后，rand1M 从「每像素遍历所有圆」的 844 ms 降到 0.538 ms，比单核 CPU 顺序参考快 712 倍。最优 tile 大小随圆半径变化：小圆场景是 $T = 8$，其余多为 $T = 16$。

目录

1 背景与作业要求 **3**

2 实验环境 **3**

3 方法与实现 **4**

3.1 Part 1：SAXPY 与传输开销 4

3.2 Part 2：六种 exclusive scan 4

3.3 Part 3：渲染器 6

*GitHub: `https://github.com/ShaneLiu04`。代码与原始指标：`https://gitee.com/liu-xingyan04/mlsys-infra`。本文基于公开课程材料独立完成，与课程团队无隶属关系。

4 实验结果 **8**
4.1 CPU/numpy 部分（已有实验回顾） 8
4.2 Part 1：SAXPY——显存带宽与 PCIe 9
4.3 Part 2：exclusive scan 11
4.4 Part 3：渲染器 14
4.4.1 (a) naive 一线程一圆是错的 14
4.4.2 (b) 正确且快：各优化阶段 16
4.4.3 分桶开销、列表长度与 tile 大小 18

5 与理论模型的对照 **20**
5.1 SAXPY：屋顶线与 PCIe 20
5.2 scan：访存遍数模型 20
5.3 渲染器的成本模型 21

6 面试专题 **21**

7 局限 **23**

8 结论 **24**

1 背景与作业要求

CS149 [11] Assignment 3 的官方 starter 是 CUDA 代码，分为三部分。本目录（`solutions/cs149/asst3-scan-render`）原先只有 numpy 版，这次补上了 GPU 部分。两者怎么对应，见表 1。

表 1: 官方作业部分与本仓库实现的对应

官方部分	本仓库 CPU/numpy 实现（已有）	本次 GPU 实现（`gpu/`）
Part 1 SAXPY	无（见 asst1 prog5）	`saxpy.cu`：kernel 带宽随 N 的变化；端到端 pinned、pageable、4-stream 分块流水；launch 开销
Part 2 `exclusive_scan`	`scan.py`：顺序、Hillis [2]–Steele、Blelloch [1]，work/span 计数	`scan.cu`：naive（starter 结构，每层一次 launch）、shared-memory Blelloch（有/无 bank padding）、raking + warp shuffle、thrust、cub
Part 2 `find_repeats`	`primitives.py`：flags → scan → scatter	flag kernel → 自写 scan → scatter，对比 `thrust::copy_if`
Part 3 circle renderer	`renderer.py`：顺序参考、乱序模拟、tile 分桶 [8] + scan 压缩 [4]	`render.cu`：CPU 顺序参考、naive 一线程一圆（错误）、每像素全圆、全局 scan 分桶、分桶 + shared 批量、块内 scan 分桶

渲染器的两个正确性约束。 三部分里最难的是渲染器，难点来自官方文档的两条要求。第一条是原子性：对同一个像素「读出颜色、与新圆混合 [7]、写回」这三步，不能被别的圆的更新打断。第二条是顺序：同一像素上的圆，必须按输入顺序混合。混合用的是 over 算子

$$\mathbf{c} \leftarrow \alpha\,\mathbf{rgb}_k + (1-\alpha)\,\mathbf{c}, \tag{1}$$

它不满足交换律，两个圆换个顺序，像素颜色就变了。所以「按圆并行，再加原子操作」也不够，还必须保序。starter 给出的 `kernelRenderCircles` 让一个线程处理一个圆，两条约束都违反了。作业真正要做的，是把并行的维度从「圆」换成「像素」或「tile」（图像上一块固定大小的方形区域），再用 scan 在不做全局排序的前提下，给每个 tile 生成一份仍按输入顺序排列的圆列表。

2 实验环境

所有 GPU 数字都在同一台机器上测得（表 2）。读后文时有两个参照值值得记住：GDDR7 显存的标称带宽 1.79 TB/s，以及 PCIe 5.0 x16 的单向标称带宽 64 GB/s，两者差了一个多数量级。

表 2: 实验环境

GPU	NVIDIA GeForce RTX 5090，32 GB（32607 MiB），170 SM，sm_120（Blackwell），L2 100663296 B（96 MB），每 SM 共享内存 102400 B，每 SM 最多 1536 线程；驱动 595.71.05；PCIe 链路 Gen5 x16（`nvidia-smi` 读到 current = max = 5，width 16）
GPU 带宽参考	GDDR7 标称 1.79 TB/s（28 Gbps × 512 bit）；同机 asst1 已测 SAXPY kernel ≈1.55 TB/s、端到端 ≈52 GB/s；PCIe 5.0 x16 单向标称 64 GB/s
主机 CPU	2 路 Intel Xeon Gold 6459C（容器可见 128 逻辑核，cgroup 限 16 核，本实验只用 `taskset -c 33,34`）；CPU 参考渲染器为单线程
工具链	CUDA 12.8 `nvcc -O3 -arch=sm_120 -std=c++17`；g++ 11.4；thrust/cub 随 CUDA 12.8；`render` 额外加 `-fmad=false`，使 device 端算术与 host 参考逐位一致
计时	`cudaEvent`，预热 2–3 次后取 5–50 次的中位数；端到端用 `steady_clock` 加同步；所有 GPU 任务都在 `flock /root/antodl-tmp/gpu.lock` 下独占运行
分析器	`ncu` 报 `ERR_NVGPUCTRPERM`（容器内无性能计数器权限，见 `ncu_attempt.txt`），改用 `cudaFuncGetAttributes` 和占用率 API 获得寄存器数与理论 occupancy
本地	Windows Server，4 物理核；Python 3.12、numpy、matplotlib；numpy 版实验与全部绘图在本地运行

原始数据在 `artifacts/raw/gpu/` 下，共七个 `.jsonl` 文件：`saxpy`、`scan`、`find_repeats`、`render`、`render_sweep`、`render_attrs`、`scan_attrs`。图像在 `artifacts/raw/gpu/images/`，其中参考图与 naive 图是 512 像素的缩略 PNG，差值图是 1024^2 全分辨率的 8-bit 图。复现只需一条命令：

```
bash gpu/build.sh && bash gpu/run_gpu.sh all && python scripts/plot_gpu.py
```

3 方法与实现

这一节按作业的三个部分，依次说明每个实验测什么、怎么测。读结果之前不必记住全部细节，但计时口径（什么算进时间、什么没算）会直接影响后面的结论。

3.1 Part 1：SAXPY 与传输开销

kernel 很简单：一个线程算一个元素 `y[i] = a*x[i] + y[i]`，每块 256 线程。我想知道的是，同一个 kernel 放进不同的「外壳」里，时间花在了哪里。因此对每个 $N = 2^{10}, 2^{12}, \ldots, 2^{28}$ 测四种情形：(a) kernel-only，数据已经在显存里；(b) pinned，主机内存用 `cudaMallocHost` 分配（页锁定，DMA 可以直接读），流程是 H2D(x, y)、kernel、D2H(y)；(c) pageable，流程相同，但主机内存是普通的 `std::vector`；(d) streams，把 pinned 内存切成 16 块，轮流放进 4 个 stream，H2D、kernel、D2H 都异步排队，这样不同块的上行和下行拷贝可以重叠。四种情形都按 $12N$ 字节（读 x、读 y、写 y）折算有效带宽。另外还测了空 kernel 连续 launch 的平均间隔，以及「launch 加同步」的往返时间。

3.2 Part 2：六种 exclusive scan

exclusive scan 的输出第 i 项是输入前 i 项之和（不含第 i 项本身）。六种实现从 starter 的写法出发，逐步减少 launch 次数和访存遍数，最后对照库实现。

naive（starter 结构）。 输入先复制到长度为 $m = 2^{\lceil \log_2 N \rceil}$ 的缓冲区，尾部补 0。up-sweep 每层一次 launch，然后把根置 0，down-sweep 也是每层一次 launch，共 $2\log_2 m$ 次 launch（测试 `test_naive_scan_launches_are_2log2m` 会逐行检查这一点）。

Listing 1: starter 结构的 up-sweep / down-sweep kernel（scan.cu）

```
__global__ void upsweep_k(int* a, long long m, long long two_d) {
  long long i = (long long)(blockIdx.x * (long long)blockDim.x + threadIdx.x) * 2 *
      two_d;
  if (i < m) a[i + 2 * two_d - 1] += a[i + two_d - 1];
}
__global__ void downsweep_k(int* a, long long m, long long two_d) {
  long long i = ...same...;
  if (i < m) { int t = a[i + two_d - 1];
            a[i + two_d - 1] = a[i + 2 * two_d - 1];
            a[i + 2 * two_d - 1] += t; }
}
```

smem / smem_nopad。 这是 GPU Gems 3 第 39 章的做法 [3]：每块 512 线程处理 1024 个元素，在 shared memory 里完成 Blelloch 的 up-sweep 和 down-sweep。每块的总和写进 `sums[]`，对 `sums` 递归调用同一过程，最后用 `uniform_add` 把块前缀加回每个元素。padding 版每 32 个元素插 1 个空位（`CF(i) = i>>5`），目的是消除深层大步长访问造成的 bank conflict（同一 warp 的多个线程访问同一个 shared memory bank，只能排队）。

Listing 2: 块内 Blelloch（节选）与递归驱动

```
for (int d = SE >> 1; d > 0; d >>= 1) { // up-sweep in shared memory
  __syncthreads();
  if (t < d) { int a = off*(2*t+1)-1, b = off*(2*t+2)-1;
            if (PAD) a += CF(a), b += CF(b);
            s[b] += s[a]; }
  off <<= 1;
}
if (t == 0) { if (sums) sums[blockIdx.x] = s[last]; s[last] = 0; }
... down-sweep symmetric ...
void Rec::run(const int* in, int* out, long long n, size_t lvl) {
  unsigned nb = (n + per_block - 1) / per_block;
  if (nb == 1) { block(in, out, nullptr, n, 1); return; }
  block(in, out, lvl_sums[lvl], n, nb); // pass 1: read N, write N
  run(lvl_sums[lvl], lvl_scan[lvl], nb, lvl + 1); // N/1024 elements
  uniform_add<<<nb, 256>>>(out, lvl_scan[lvl], n, per_block); // pass 2: read N,
      write N
}
```

raking。 每块 256 线程处理 4096 个元素。先用合并访问把数据装进 shared memory（同样加 padding），每个线程在寄存器里顺序扫自己的 16 个元素，再用 `__shfl_up_sync` 在 warp 内做 inclusive scan，最后由第 0 个 warp 扫 8 个 warp 的和。递归结构与 smem 相同，但每块的元素数是它的 4 倍，所以递归层数和 block 数都更少。

thrust / cub。 `thrust::exclusive_scan(thrust::device, ...)` 每次调用都在内部临时分配缓冲区；`cub::DeviceScan::ExclusiveSum` 则使用预先分配好的临时空间。两者底层都是 Merrill–Garland 的单遍 decoupled look-back scan[5]，每个元素只读一次、写一次。

测试数据与正确性。 输入取 `hash32(i) & 7`，这样 2^{28} 个元素的总和也小于 2^{31}，不会溢出。每个 $(N, \text{算法})$ 组合都把整个输出拷回主机，与 CPU 顺序 scan 逐元素比较，不一致的个数写进

`scan.jsonl`。N 取 19 个值，特意混入了非 2 的幂：10^3、2^{10}、4096、10^4、2^{16}、10^5、2^{18}、2^{20}、$2^{20}+1$、10^6、3000007、2^{22}、10^7、$2^{24}-1$、2^{24}、2^{26}、10^8、2^{27}、2^{28}。

find_repeats。 任务是找出所有满足 `a[i]==a[i+1]` 的下标。自写版本分三步：`repeat_flags` 计算标志 `f[i] = a[i]==a[i+1]`；用自写 scan（smem 或 raking）把标志变成输出位置；`repeat_scatter` 按位置写出下标，scatter 时重新计算一次谓词，省掉再读一遍 flags。因为 `f[n-1]=0`，输出个数就是 `pos[n-1]`，需要一次 4 字节的 D2H。对照组是 `thrust::copy_if(counting_iterator, …, IsRepeat{a})`。输入取值范围为 $[0,100)$ 和 $[0,2)$，分别对应约 1% 和 50% 的重复率。

3.3 Part 3：渲染器

场景。 图像是 1024×1024，背景为白色，坐标归一化到 $[0,1]$。所有场景都由固定种子的 xorshift 生成，见表 3。表中的「平均覆盖」是 $\sum_k \pi r_k^2$，也就是平均每个像素被多少个圆盖住（overdraw）。它决定了渲染的基本工作量，后文会反复用到。snow 场景按 z 从远到近排序，颜色和 α 随到圆心的归一化距离衰减：$\alpha = 0.5\,\mathrm{clamp}(0.6+0.4(1-z))\,e^{-4d^2}$。其他场景都取 $\alpha=0.5$，与官方 starter 一致。

表 3: 场景定义（`render.cu: make_scene`）

场景	N	平均覆盖	说明
rgb	3	0.8	三个大圆（$r=0.3$）互相重叠
rand10k	10000	44.9	$r\sim U(0.01,0.06)$
rand100k	100000	112.6	$r\sim U(0.005,0.03)$
biglittle	50020	10.4	20 个大圆（$r\in[0.15,0.35]$）在底层，5 万个小圆在上层
littlebig	50020	10.7	同上，顺序相反：大圆在最上层
pattern	5120	1.9	32^2 与 64^2 两层规则网格，半径按正弦变化
snowsingle	50000	39.2	雪花着色（径向 α 衰减、`sqrtf/expf`），按深度排序
rand1M	1000000	59.7	$r\sim U(0.001,0.007)$
micro2M	2000000	6.8	$r\sim U(0.0005,0.0015)$，半径约 0.5–1.5 像素
sweep_N	10^3–3×10^6	≈45	rand10k 的半径乘 $\sqrt{10^4/N}$，overdraw 保持不变

CPU 参考。 正确性要有一个可信的基准。CPU 参考按输入顺序遍历圆，对每个圆的保守包围盒（`floor((cx-r)W)` 到 `ceil((cx+r)W)`）内的每个像素中心做 `d2 <= r*r` 测试，命中就按式 (1) 混合。像素中心取 $((x+0.5)/W,(y+0.5)/H)$。host 和 device 共用同一个 `__host__ __device__ shade()`，并用 `-fmad=false` 禁止编译器把乘加合并成 FMA。有了这两条，$\alpha=0.5$ 的场景就可以要求 GPU 与 CPU 逐位相等，而不是「误差足够小」。

五个 GPU 阶段。 下面五个版本从错误的 starter 写法开始，每一步改掉上一步的一个主要问题。

1. naive_per_circle：starter 的 `kernelRenderCircles`。一个线程负责一个圆，对包围盒内的像素做非原子的「读、shade、写」。
2. allcircles：一个线程负责一个像素（16×16 的块），像素颜色放在寄存器里，按顺序遍历全部 N 个圆，最后写一次。它是正确的，但工作量为 $O(NP)$（P 为像素数）。
3. binned：全局分桶。`k_bin_count` 算出每个圆覆盖几个 tile（`cnt[k]`），同时用 `atomicAdd` 累加每个 tile 的计数。对 `cnt` 做 exclusive scan，得到每个圆的写入偏移；对 tile 计数做 exclusive scan，得到每个 tile 列表的起点 `tile_start`。`k_bin_emit` 按偏移写出 (tile, circle) 对，这时数

组是按圆的下标排列的。最后用 `cub::DeviceRadixSort::SortPairs` 只对 $\lceil \log_2(\#\text{tiles}) \rceil$ 位 key 排序。radix sort 是稳定的，相同 key 保持原来的相对顺序，所以同一 tile 内的圆仍按输入顺序排列。渲染时一个 block 对应一个 tile，每个线程负责一个像素，从 global memory 逐个读圆的参数。

4. binned_smem：圆列表与 binned 相同，区别在渲染时怎么取圆。block 每次协作把 `blockDim` 个圆的 7 个参数（SoA 布局）装进 shared memory，同步之后每个像素遍历这一批圆。所有线程读的是同一地址，属于 broadcast，不会有 bank conflict。

5. blockscan_smem：不生成全局列表，这是 CS149 常见的参考解法。每个 block 每轮取 `blockDim` 个圆，每个线程检查其中一个圆的包围盒是否与本 tile 相交；对命中标志做一次 block 级 exclusive scan，得到压缩后的位置，命中的圆按原顺序写进 shared memory，然后所有像素着色这一批圆。整个过程只有一次 launch，代价是每个 tile 都要测试全部 N 个圆。

下面两段代码分别是 blockscan 的块内循环和全局分桶的主机端流程，可以对照着看两种做法在哪里生成圆列表。

Listing 3: 块内 scan 分桶（`k_render_blockscan`，节选）

```
for (int cb = 0; cb < s.n; cb += bs) {
  int k = cb + tid, hit = 0;
  if (k < s.n) { int x0, x1, y0, y1;
    circ_bbox(s.x[k], s.y[k], s.r[k], W, H, x0, x1, y0, y1);
    hit = x0 <= tx1 && x1 >= tx0 && y0 <= ty1 && y1 >= ty0; }
  int total, pos = block_excl_scan(hit, ws, total); // warp shuffle + warp sums
  if (hit) b.put(pos, s, k); // compacted, still in input order
  __syncthreads();
  for (int i = 0; i < total; ++i)
    shade(b.x[i], b.y[i], b.z[i], b.r[i], b.R[i], b.G[i], b.B[i], s.snow, fx, fy, c)
        ;
  __syncthreads();
}
```

Listing 4: 全局分桶的主机端流程（`Binner::build`，节选）

```
k_bin_count<<<...>>>(s, W, H, T, TX, cnt, tcnt); // per-circle tile count + per-
    tile atomics
cub::DeviceScan::ExclusiveSum(tmp, b, cnt, off, s.n); // circle -> pair offset
cub::DeviceScan::ExclusiveSum(tmp, b, tcnt, tstart, nt+1); // tile -> list start
cudaMemcpy(&last, off + s.n - 1, ...); // total pairs (sync)
k_bin_emit<<<...>>>(s, W, H, T, TX, off, k1, v1); // (tile, circle) in circle order
cub::DeviceRadixSort::SortPairs(tmp, b, k1, k2, v1, v2, pairs, 0, bits); // stable
```

计时口径。 每帧时间包含清屏 kernel。binned 系列的时间包含完整的分桶流程（2 次 scan、1 次排序、2 次 4 字节 D2H 同步），另外单独记录 `bin_ms` 和 `render_ms`，方便拆开看。正确性检查对每个 (场景, 阶段, tile) 都把整帧拷回，计算与 CPU 参考的最大绝对误差，并统计误差 $> 10^{-6}$ 和 $> 10^{-3}$ 的像素数。naive 阶段额外连续跑两次，统计两次结果不同的像素数，用来区分「确定性的错」和「竞争造成的随机错」。

4 实验结果

4.1 CPU/numpy 部分（已有实验回顾）

看 GPU 结果之前，先回顾原有的三张 numpy 图。它们给的是与硬件无关的计数：加法做了多少次，乱序会错多少像素，分桶能省多少次测试。后面 GPU 上的很多现象，都能在这里找到对应。数据由原有的 `scripts/run_experiments.py` 生成，存在 `artifacts/raw/*.csv`。

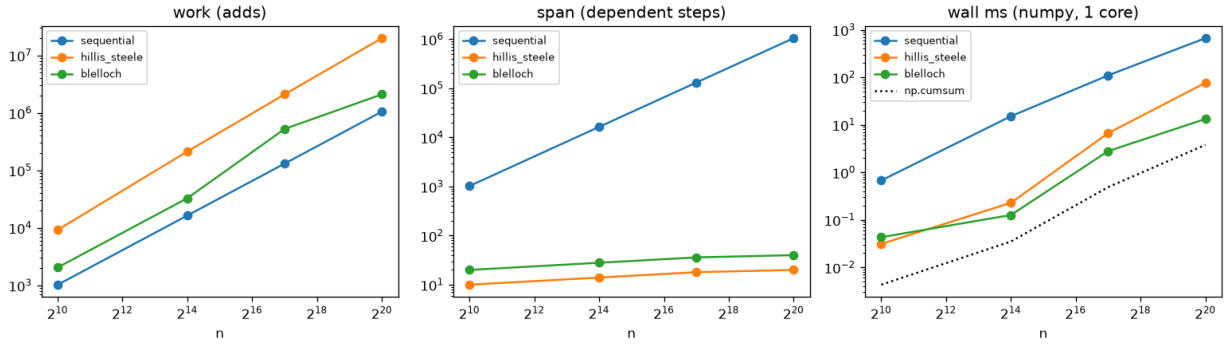


图 1: numpy 版 scan 的 work（加法次数）、span（相互依赖的步数）和单核墙钟时间。

图 1 说明 work 和 span 是两笔不同的账。Blelloch 的 work 恒为 $2(m-1)$，Hillis–Steele 是 $\Theta(n \log n)$。$n = 2^{20}$ 时两者分别做 2.10M 和 19.9M 次加法，单核 numpy 墙钟是 13.2 ms 和 76.5 ms，在单核上 work 少就是快。图中还能看到补齐的代价：$n = 2^{17}+1$ 要补到 2^{18}，Blelloch 每个元素的加法次数（adds/n）从 2 变成 4，GPU 上的 naive 版本也要付这笔账（图 8）。span 一列则预告了 GPU 上的另一笔成本：naive 版需要 $2\log_2 m$ 次全局同步，而在 GPU 上，每次全局同步就是一次 kernel launch。

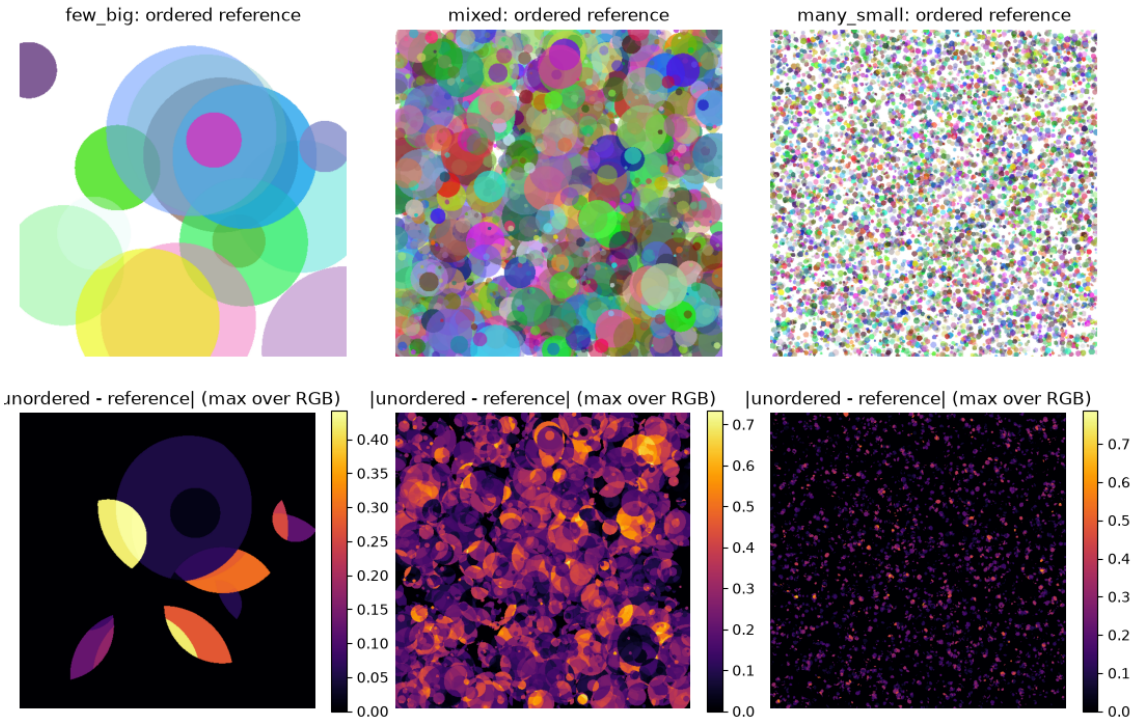


图 2: numpy 版：随机打乱圆的顺序（模拟按圆并行、不保序）与顺序参考之间的差。

图 2 回答「只是顺序乱了，会错多少」。在 512 像素的 mixed 场景（2000 个圆）中，打乱顺序后有 92%–93% 的像素误差超过 0.01。出错的只是那些被至少两个圆覆盖、而这些圆的相对顺序又被打乱了的像素，可这样的像素已经占了绝大多数。GPU 上 naive kernel 的错误比这还严重（第 4.4.1 节），因为它除了顺序错，还有读–改–写竞争。



图 3: numpy 版：tile 分桶相对 brute force（每像素测试所有圆）省下的测试次数。

图 3 显示分桶能省多少工作，以及 tile 大小为什么存在一个最优点。many_small（8000 个圆）在 tile=16 时，测试次数比 brute force 少 167 倍。tile 太小时，包围盒测试（圆数 × tile 数）占主导；tile 太大时，每个 tile 的圆列表又变长。GPU 上的 tile 扫描（图 17）得出同样的结论，只是还要再加上 launch、排序和 shared memory 容量的约束。

4.2　Part 1：SAXPY——显存带宽与 PCIe

SAXPY 每个元素只做一次乘加，却要搬 12 个字节，是最典型的访存受限 kernel。正因为计算几乎可以忽略，它能把 GPU 上的几层「墙」一次看清：launch 延迟、L2、显存和 PCIe。

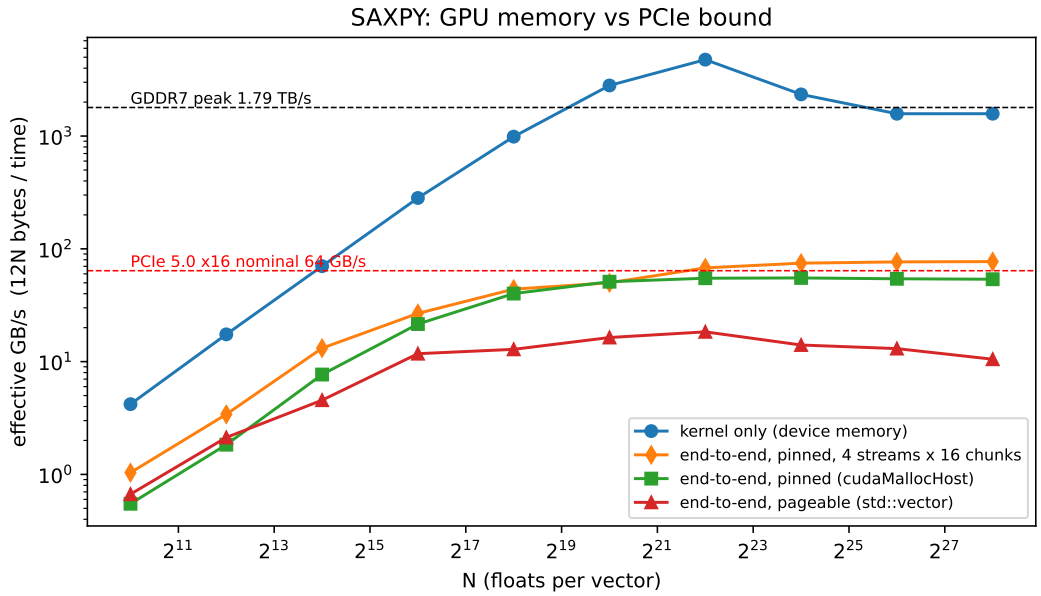


图 4: SAXPY 有效带宽随 N 的变化（`saxpy.jsonl`）。虚线分别为 GDDR7 标称带宽和 PCIe 5.0 x16 单向标称带宽。

从左往右看，图 4 依次经过四个区间，每个区间被不同的东西卡住：

- $N \le 2^{14}$ 时由 launch 延迟主导。kernel 时间几乎不变（2^{10} 时 2.93 μs，2^{16} 时 2.78 μs），所以带宽与 N 成正比地上升。作为参照，空 kernel 连续 launch 的间隔实测为 1.85 μs，launch 加同步的往返为 5.72 μs。
- 2^{20} 到 2^{24} 之间，数据装得进 L2。$N = 2^{22}$ 时 kernel 「达到」4759 GB/s，是 GDDR7 标称值的 2.7 倍。这不是显存变快了：工作集 $12N = 50$ MB 小于 96 MB 的 L2，而前面的 `cudaMemcpy` 刚好把数据留在了 L2 里，所以这里量到的是 L2 带宽。
- $N \ge 2^{26}$ 时才真正由显存带宽决定。2^{26} 和 2^{28} 时 kernel 分别为 1577 和 1578 GB/s，是 1.79 TB/s 标称值的 88%，与 asst1 测得的约 1.55 TB/s 一致。
- 端到端的曲线则由 PCIe 决定。2^{28} 时 pinned 端到端为 53.8 GB/s，其中 H2D 56.5 GB/s、D2H 54.1 GB/s，约为 PCIe 5.0 x16 标称 64 GB/s 的 85%–88%。pageable 只有 10.5 GB/s（H2D 10.8 GB/s），因为驱动得先用 CPU 把数据拷进内部的 pinned staging 缓冲区，再做 DMA，瓶颈落在主机端的 memcpy 上。streams 版达到 77.1 GB/s，超过了单向 PCIe 带宽。原因是 PCIe 全双工，H2D 和 D2H 可以同时跑。它用时 41.8 ms，接近单独 H2D 的 38.0 ms，也就是说，流水化之后的时间下限是较长那个方向的拷贝时间。

带宽曲线只告诉我们「慢」，下面的时间分解告诉我们慢在哪一段。

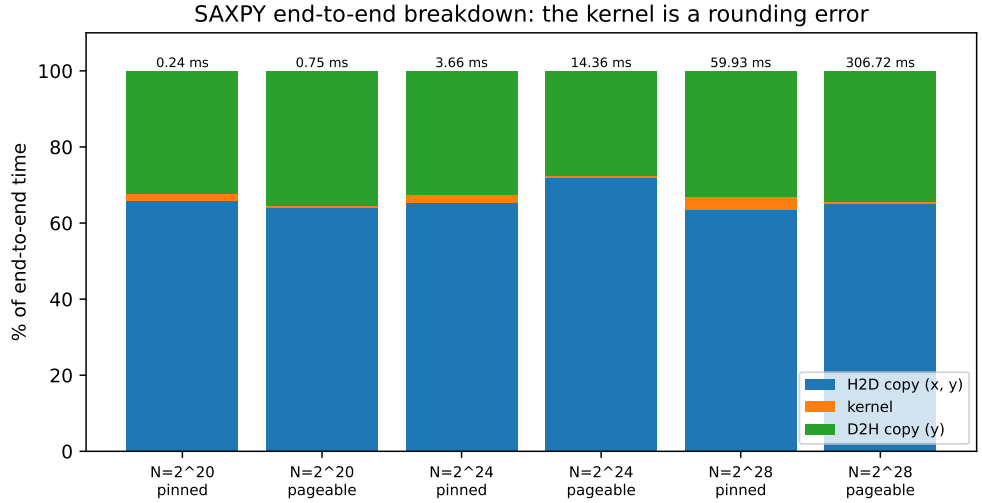


图 5: 端到端时间分解（各段取中位数）。顶部数字为三段之和。

图 5 中，$N = 2^{28}$、pinned 时，H2D 用 38.04 ms，kernel 2.04 ms，D2H 19.85 ms，kernel 只占 3.4%。H2D 要搬 x 和 y 共 8N 字节，D2H 只搬 4N 字节，所以 H2D 的时间约为 D2H 的两倍。pageable 下总时间为 306.7 ms，是 pinned 的 5.1 倍。

SAXPY 的算术强度（每搬一个字节做多少次浮点运算）只有 $2/12 \approx 0.17$ FLOP/B，在任何 GPU 上都受带宽限制。由此得到这一部分的结论：

结论：SAXPY

kernel 本身已经用到 88% 的显存带宽；一旦数据要经 PCIe 往返，瓶颈就换成了比显存慢约 29 倍（1578 / 53.8）的 PCIe。对这类 kernel，GPU 有收益的前提是数据常驻显存，或者拷贝之后有足够多的计算来摊薄传输。

4.3 Part 2：exclusive scan

scan 的 work 很容易算清楚，可 GPU 上的快慢并不按 work 排序。这一节想弄明白：六种实现之间的差距，到底是被什么决定的？

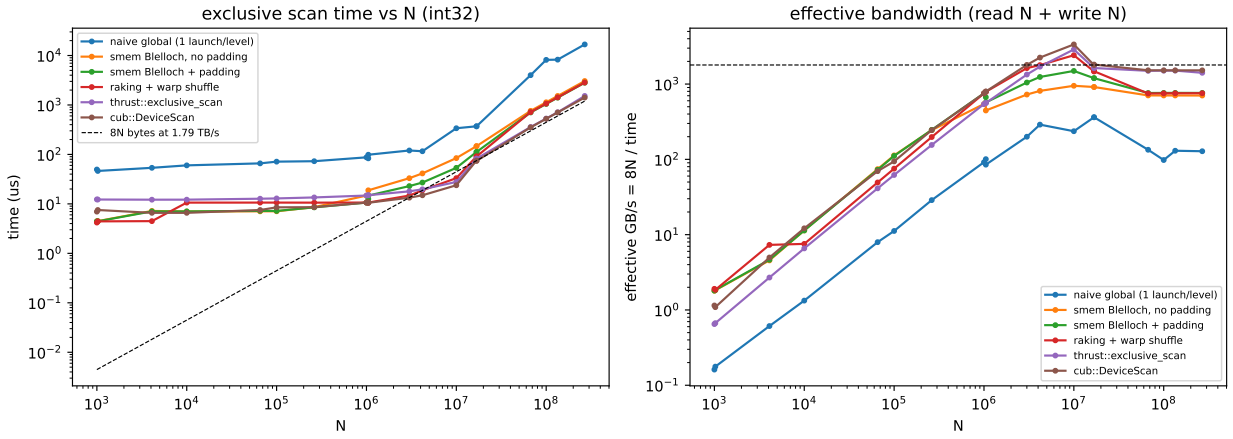


图 6: 六种 scan 的时间（左）和有效带宽 $8N/t$（右）。所有点 mismatch = 0。

图 6 可以按 N 分成三段来读。

大 N（$\geq 2^{26}$）时，排名很清楚。2^{28} 时 cub 用 1.412 ms（1521 GB/s），thrust 1.522 ms，smem 2.817 ms（762 GB/s），raking 2.839 ms，smem_nopad 3.049 ms，naive 16.75 ms（128 GB/s）。有意思的是，smem 和 raking 的有效带宽几乎正好是 cub 的一半。原因在流量：它们第一遍读 N、写 N，`uniform_add` 再读 N、写 N，实际搬了 $16N$ 字节。按 $16N$ 重新折算，它们是 1.52 TB/s，与 cub 的 1.52 TB/s 相同。换句话说，三者都已经跑在显存带宽的上限，差别只在搬了多少字节。

中等 N（2^{22}–2^{24}）时，数据有一部分留在 L2 里，有效带宽可以超过显存：cub 在 2^{24} 时为 1816 GB/s，raking 为 1479 GB/s。

小 N 时，时间不再随 N 变化。这一段由固定开销决定，下面单独放大来看。

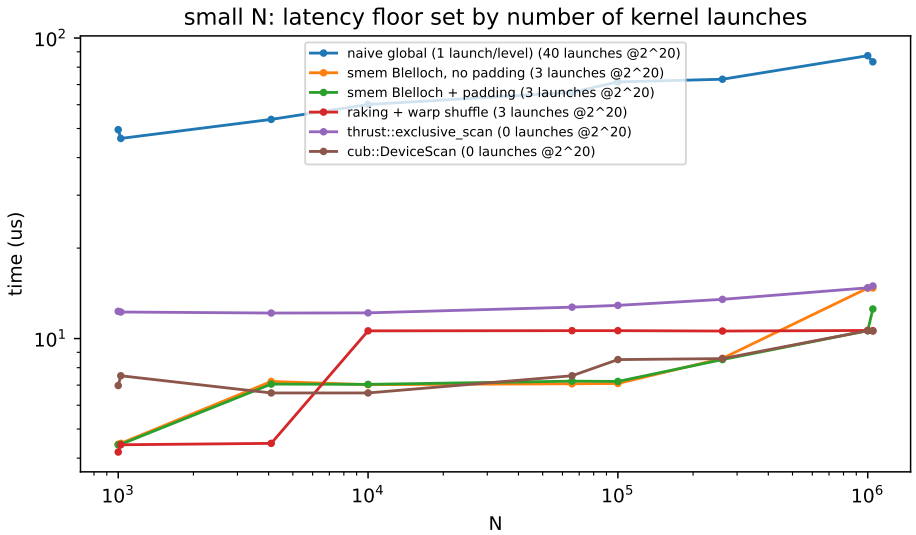


图 7: $N \leq 2^{20}$ 时的延迟下限。图例中给出 $N = 2^{20}$ 时各算法的 kernel launch 数（thrust/cub 内部的 launch 未计入）。

图 7 中，各算法的时间基本可以用「launch 次数乘以每次的开销」来解释。$N = 1000$ 时，naive 需要 20 次 launch，外加一次 memcpy 和两次 memset，用时 49.55 μs，平均每次操作约 2 μs，正好

与实测的 1.85 μs 连续 launch 间隔对得上。单 block 的 smem 和 raking 只需一次 launch，用时 4.4 μs；cub 要两次 launch（初始化 tile 状态，再做 scan），用时 6.98 μs；thrust 每次调用都要分配和释放临时缓冲，用时 12.3 μs。N 从 1000 增加到 2^{20}，涨了 1000 倍，smem 版的时间却只从 4.43 变到 12.54 μs，因为这段时间大部分是固定开销。

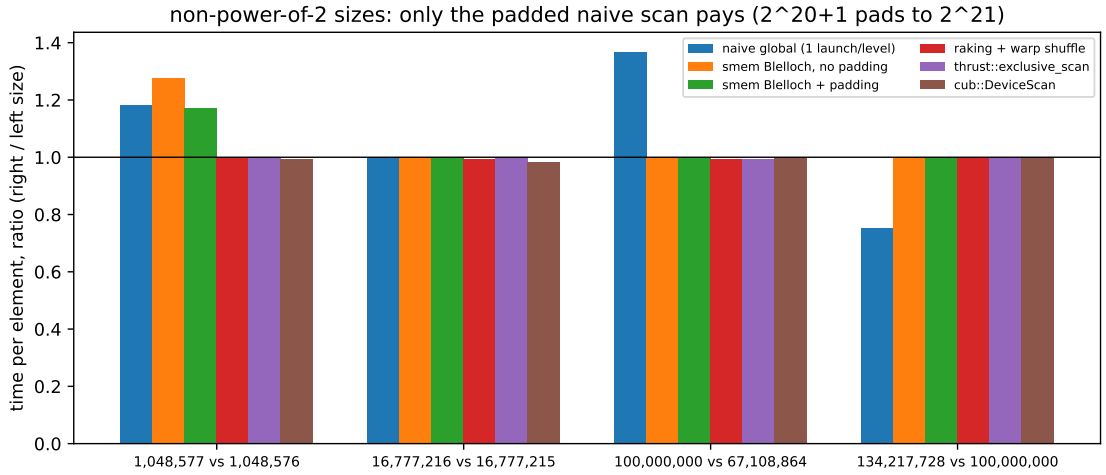


图 8: 非 2 的幂长度：每元素时间之比（右侧规模 / 左侧规模）。

非 2 的幂长度检验的是补齐的代价。图 8 中，naive 必须补齐到 2 的幂：10^8 要补到 2^{27}，于是每元素时间是 2^{26} 的 1.37 倍，与补齐比例 $2^{27}/10^8 = 1.34$ 吻合。反过来，2^{27} 与 10^8 的补齐长度相同，所以 2^{27} 的每元素时间只有 10^8 的 0.75 倍。分块算法不需要补齐，比值都在 1.0 左右。唯一的例外是 $2^{20}+1$：smem 每块 1024 个元素，多出的 1 个元素会多出一个 block，块和数组的长度从 1024 变成 1025，于是多了一层递归（launch 数从 3 变为 5）。在这个规模下，多出的约 2 μs 很显眼。raking 每块 4096 个元素，块和只有 257 个，不受影响。

既然大 N 时差距都能用流量解释，那么 occupancy 和 bank conflict 起了多大作用？表 4 给出各 kernel 的静态资源。

表 4: scan kernel 的静态资源与理论 occupancy（`scan_attrs.jsonl`；sm_120 每 SM 最多 1536 线程）

kernel	block	寄存器	静态 smem (B)	blocks/SM	理论 occupancy
upsweep_k	256	10	0	6	1.00
blelloch_block（pad / nopad）	512	18	4224	3	1.00
raking_block	256	64	16928	4	0.67
uniform_add	256	14	0	6	1.00

raking 每个线程在寄存器里放 16 个元素，用到 64 个寄存器，理论 occupancy（SM 上实际驻留的线程数占上限的比例）降到 0.67。可它在 2^{28} 时与 smem 一样快（2.839 和 2.817 ms），说明这里的瓶颈不在 occupancy，而在 DRAM。padding 版只多用 128 B 的 shared memory（两种版本都按 4224 B 静态分配），却在 2^{24} 时把时间从 146.9 降到 112.1 μs，快了 1.31 倍。由于 ncu 不可用，bank conflict 的次数没有直接计数，这个 1.31 倍是唯一的证据。

结论：scan

大规模下，scan 的快慢由读写显存的遍数决定，与 work 是 $2N$ 还是 N 关系不大：分块递归版把流量降到 $16N$，单遍的 cub 降到 $8N$。小规模下，决定时间的是 launch 次数，每次约 2 μs。

scan 最常见的用途是流压缩（stream compaction），`find_repeats` 就是一例。它能检验上面的结论：如果遍数决定一切，那么多遍的自写版本应该输给单遍的库实现。

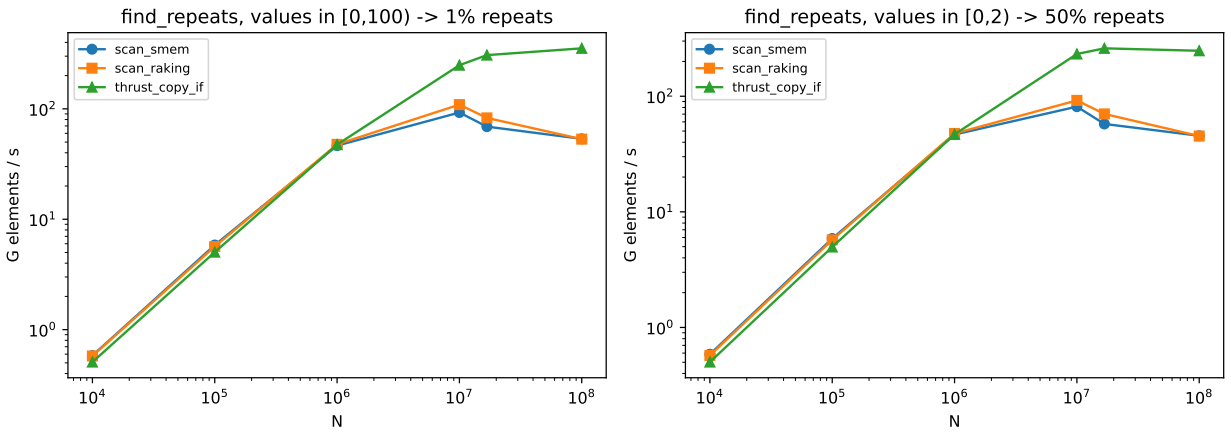


图 9: `find_repeats` 吞吐量（`find_repeats.jsonl`），左：约 1% 重复，右：约 50% 重复。所有结果与 CPU 一致。

图 9 的结果正是这样。$N \le 10^6$ 时三者都在约 17–21 μs，由 launch 和同步决定：自写版本要发 5 个以上 kernel 再做一次 4 字节 D2H，thrust 也要 D2H 取回输出个数。到了 $N = 10^8$、1% 重复时，自写 smem 版用 1.869 ms，thrust 用 0.283 ms，后者快 6.6 倍。算一下自写版本的流量：flags 读 a 写 f（$8N$），scan（$16N$），scatter 读 a 和 pos（$8N$），共约 $32N = 3.2$ GB。按 1.87 ms 算是 1.71 TB/s，已经贴近显存带宽上限 [9]。所以自写版本并不是哪一步写得慢，而是遍数太多。`copy_if` 用单遍 look-back，把谓词计算和压缩合在一个 kernel 里，基本只读一次 a（$4N$），流量少约 8 倍。重复率升到 50% 时，thrust 从 0.283 ms 增加到 0.404 ms，因为要多写 $0.5N$ 个输出；自写版从 1.869 增加到 2.198 ms。

把所有访存受限的 kernel 放在同一根轴上，能更直观地看出「离上限有多远」。

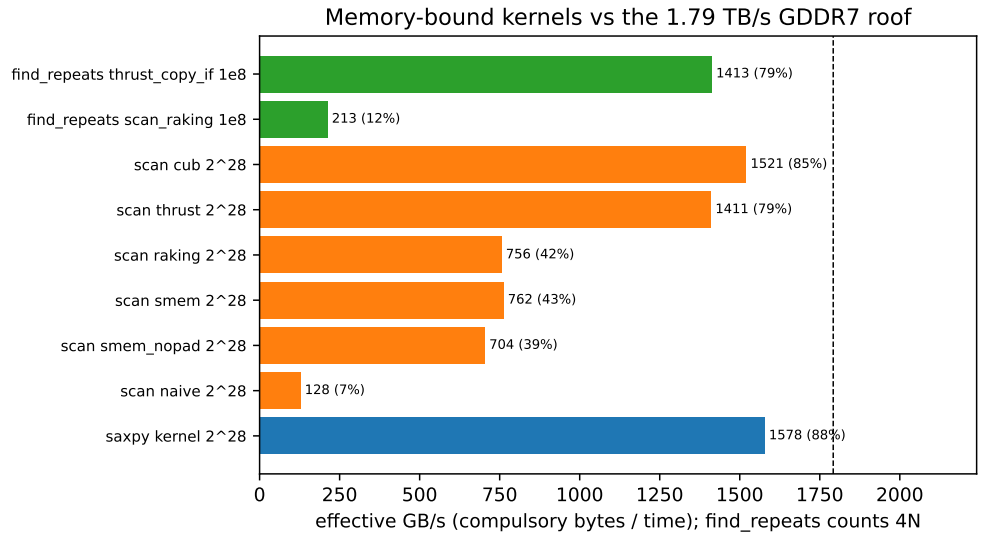


图 10: 访存受限 kernel 与 1.79 TB/s 上限对比（按算法必需的字节数计：scan 为 $8N$，find_repeats 为 $4N$）。

图 10 按「算法必需的字节数除以时间」计算带宽。SAXPY 到 88%，cub 85%，thrust scan 79%，thrust copy_if 79%；自写的 smem 和 raking 只有 42%–43%（nopad 39%），因为实际流量是必需流量

的两倍；naive 只有 7%。在这根轴上，离上限的距离就是多出来的访存遍数。

4.4 Part 3：渲染器

渲染器要同时满足正确和快。这一节先确认 starter 的写法错在哪里，再看各个正确版本是怎样一步步变快的，最后讨论 tile 大小怎么选。图 11 是 CPU 参考的渲染结果，后面所有对比都以它为准。

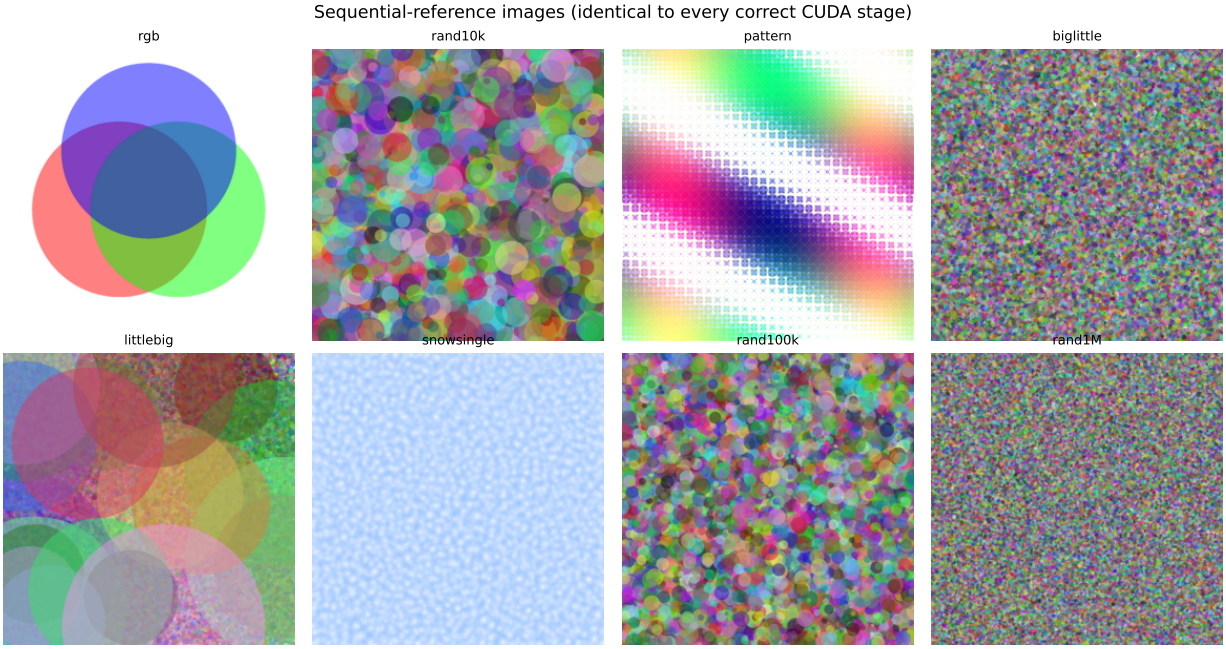


图 11: CPU 顺序参考渲染结果（512 像素缩略图）。所有正确的 GPU 阶段与之逐位相同（snow 场景差 $\leq 2.98 \times 10^{-7}$）。

4.4.1 (a) naive 一线程一圆是错的

先看错误长什么样。图 12 把参考图、naive 的输出和两者之差并排放在一起，图 13 和表 5 给出定量统计。

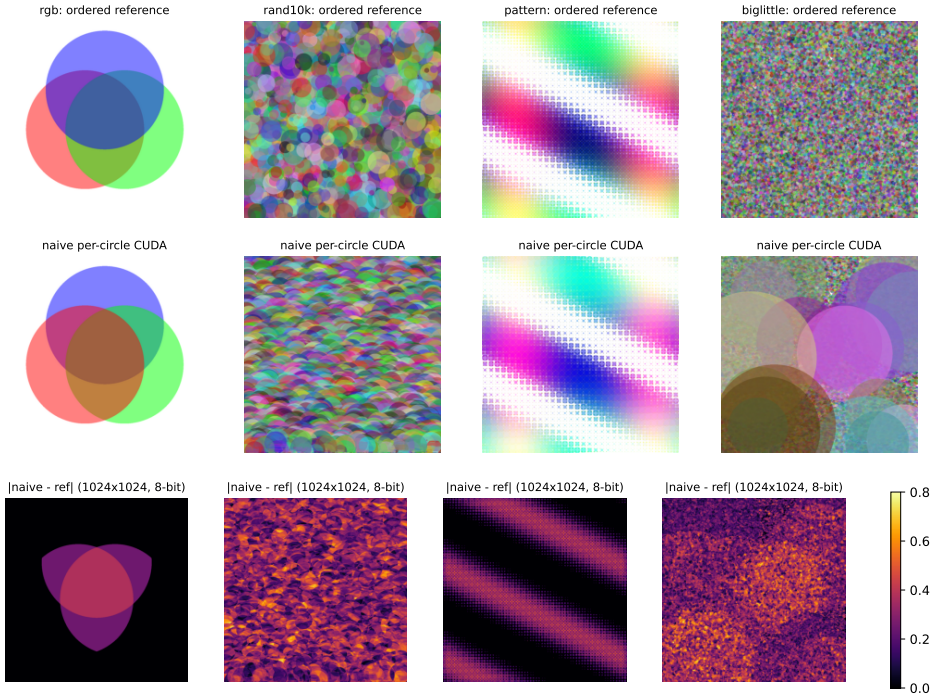


图 12: 上：顺序参考；中：naive CUDA；下：全分辨率逐像素最大通道差（8-bit 量化）。

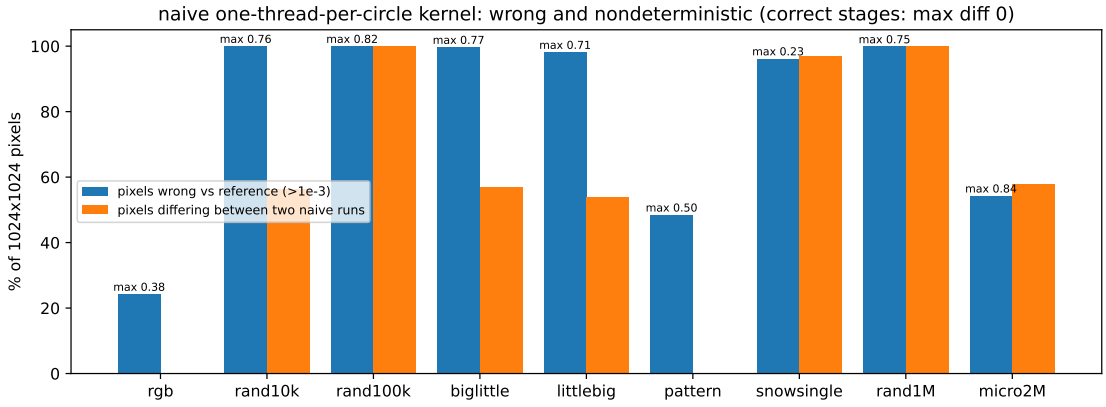


图 13: naive kernel 的出错像素比例，以及两次运行结果不同的像素比例（`render.jsonl`）。

表 5: naive 一线程一圆 kernel 的错误统计（共 1048576 个像素）

场景	最大绝对误差	像素 $> 10^{-6}$	像素 $> 10^{-3}$	两次运行不同的像素
rgb	0.375	254534	254534	0
rand10k	0.764	1048576	1048576	591059
rand100k	0.817	1048576	1048576	1048576
biglittle	0.767	1045372	1045372	597239
littlebig	0.710	1029479	1029381	564817
pattern	0.505	508748	508334	1462
snowsingle	0.232	1048515	1008719	1017242
rand1M	0.754	1048385	1047349	1048518
micro2M	0.840	589547	569401	608102

这些数据里能分出两种性质不同的错误。

第一种只有顺序错，结果却是确定的，rgb 场景就是这样。三个圆的线程在同一个 warp 里，三条 lane 以 lockstep 方式各自遍历自己的包围盒。同一个像素会在不同的循环迭代里被不同的圆访问，谁先谁后由包围盒的几何形状决定，与输入顺序无关。结果是 24.3% 的像素出错，但两次运行完全一样（0 个像素不同）。这个例子提醒我们：结果可复现，并不代表结果正确。

第二种是读–改–写竞争。其余场景中，不同 warp、不同 SM 会同时读写同一像素，有的更新被覆盖丢失，有的顺序随机，rand100k 两次运行时所有像素都不同。biglittle 的 naive 图里，大圆反而浮在了最上层（图 12 第 4 列）：大圆对应的线程循环最长，最后才写完，它们的写入就盖住了小圆。这与输入顺序（大圆在底层）正好相反。

pattern 场景介于两者之间。它的重叠很少（平均覆盖 1.9），只在圆有重叠的斜带上出错：48.5% 的像素错，但两次运行只有 1462 个像素不同。可见它的错误大部分来自确定的顺序，只有少量来自竞争。

4.4.2 (b) 正确且快：各优化阶段

确认了必须按像素并行之后，问题变成：怎样让每个像素只看真正覆盖它的圆？表 6 汇总了各阶段的每帧时间。

表 6: 每帧时间（ms，取各阶段在不同 tile 下的最优值，括号内为 tile 大小）。加速比为 CPU 参考除以最快的正确 GPU 阶段。

场景	N	覆盖	CPU	naive（错误率）	allcircles	binned	binned_smem	blockscan	加速比
rgb	3	0.8	7.4	46.518 (24.3%)	0.015	0.049 (32)	0.047 (32)	0.015 (16)	506
rand10k	10000	44.9	129.9	2.837 (100.0%)	8.322	0.165 (16)	0.136 (16)	0.150 (32)	952
rand100k	100000	112.6	422.7	2.782 (100.0%)	84.532	0.411 (16)	0.303 (16)	1.008 (32)	1393
biglittle	50020	10.4	45.4	58.206 (99.7%)	41.930	0.141 (16)	0.121 (16)	0.404 (32)	375
littlebig	50020	10.7	45.4	52.800 (98.2%)	41.927	0.140 (16)	0.120 (16)	0.406 (32)	378
pattern	5120	1.9	6.0	0.838 (48.5%)	4.263	0.074 (16)	0.070 (16)	0.047 (32)	126
snowsingle	50000	39.2	393.3	1.841 (96.2%)	22.472	0.258 (16)	0.201 (16)	0.533 (32)	1957
rand1M	1000000	59.7	383.3	2.109 (99.9%)	844.282	0.587 (8)	0.538 (16)	7.591 (32)	712
micro2M	2000000	6.8	147.8	0.638 (54.3%)	1690.730	0.463 (8)	0.505 (8)	14.239 (32)	319

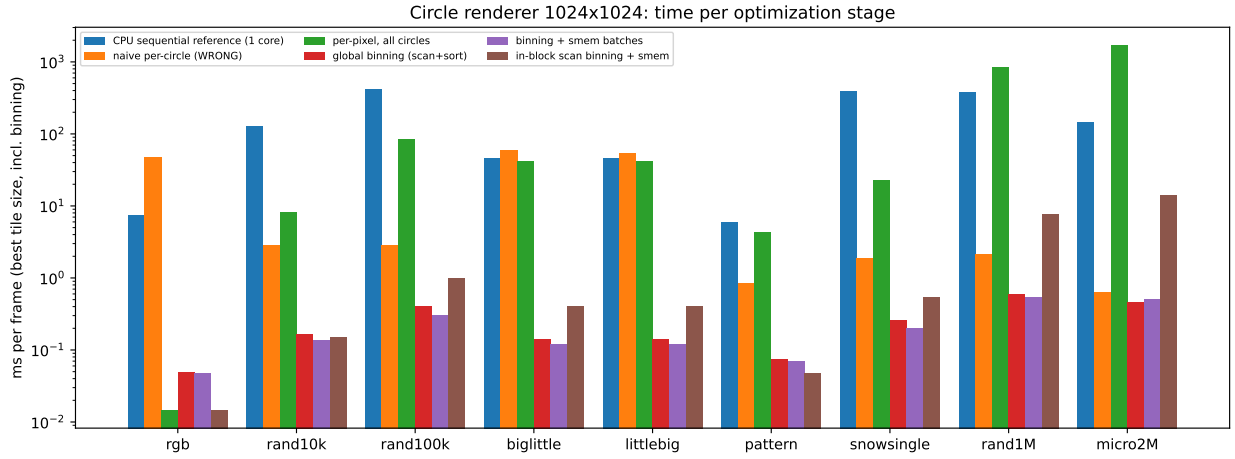


图 14: 各优化阶段的每帧时间（对数轴，最优 tile，包含分桶时间）。

沿着表 6 和图 14 从左往右，每一步都对应一个具体的改进。

allcircles 是正确的，但工作量是 $O(NP)$。rand1M 要做 1.05×10^{12} 次圆–像素测试，用时 844 ms，约合每秒 1.24×10^{12} 次测试，比单核 CPU 参考（383 ms）还慢。这是因为 CPU 参考只测包围盒内的像素，工作量与覆盖面积成正比，而不是与 NP 成正比。

binned 把测试次数降到 $P \times \overline{L}$，其中 $\overline{L}$ 是每个 tile 的平均列表长度。rand1M 在 $T = 16$ 时 $\overline{L} = 609.5$，只需 6.39×10^8 次测试，是 allcircles 的 1/1640。它的 render kernel 用 0.385 ms，约合每秒 1.66×10^{12} 次测试。单次测试的吞吐与 allcircles 差不多，所以加速几乎全部来自测试次数的减少。

binned_smem 在此基础上改用 shared memory 批量取圆，在 $T \geq 16$ 时有效：rand100k $T = 32$ 的 render 时间从 0.502 降到 0.293 ms（1.71 倍），rand1M $T = 32$ 从 1.772 降到 0.849 ms（2.09 倍）。收益来自两处。一是每个圆的 7 个参数，每个 block 只从 global memory 读一次；二是寄存器从 52 个降到 34 个（表 7），256 线程块的理论 occupancy 从 0.67 升到 1.0。$T = 8$ 时每批只有 64 个圆，同步开销抵掉了收益。

blockscan 只有一次 launch，也不需要全局排序，所以在 N 很小的场景里最快（rgb 0.015 ms，pattern 0.047 ms）。但它的工作量是「tile 数 $\times N$」次包围盒测试：micro2M 在 $T = 16$ 时要做 $4096 \times 2 \times 10^6 = 8.2 \times 10^9$ 次，用时 28.8 ms，比 binned 慢 30 倍以上。

表格里的场景各不相同，不容易分清每种方法随 N 怎么变。下面的 sweep 固定 overdraw、只改变 N，把渐近行为单独拿出来看。

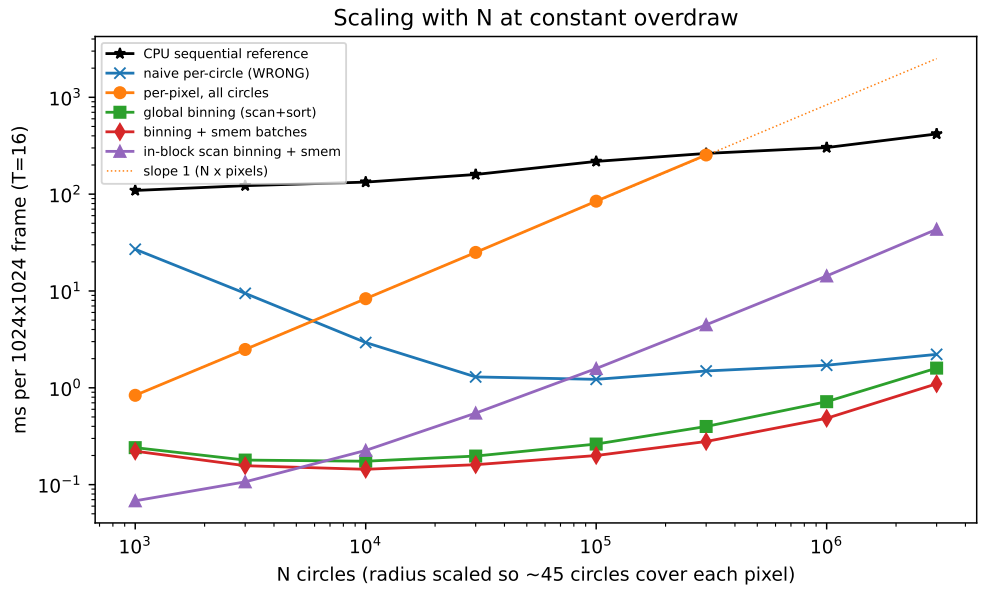


图 15: overdraw 固定为约 45、N 从 10^3 到 3×10^6（`render_sweep.jsonl`，$T = 16$）。allcircles 在 $N > 3 \times 10^5$ 时不再运行。

图 15 中，allcircles 的斜率为 1：N 每增加 10 倍，时间也增加约 10 倍（10^4 时 8.33 ms，10^5 时 84.6 ms），到 3×10^5 时已与 CPU 相当（253.8 和 263.3 ms）。blockscan 的斜率也接近 1，因为它测试的是「tile 数 $\times N$」。binned_smem 则几乎是平的：N 增加 3000 倍，它只从 0.222 ms 增加到 1.101 ms。原因是总覆盖面积不变，渲染工作量基本不变；增加的只是分桶开销，以及圆变小后跨 tile 的包围盒占比上升带来的更长列表。两条曲线在 $N \approx 5000$ 处相交：$N = 3000$ 时 blockscan 更快（0.107 和 0.157 ms），$N = 10^4$ 时 binned_smem 更快（0.144 和 0.226 ms）。CPU 参考的时间与 N 关系不大（109–418 ms），因为它的工作量同样与覆盖面积成正比。

4.4.3 分桶开销、列表长度与 tile 大小

分桶本身也要花时间。对圆很少的场景，这笔固定开销可能比渲染还贵。

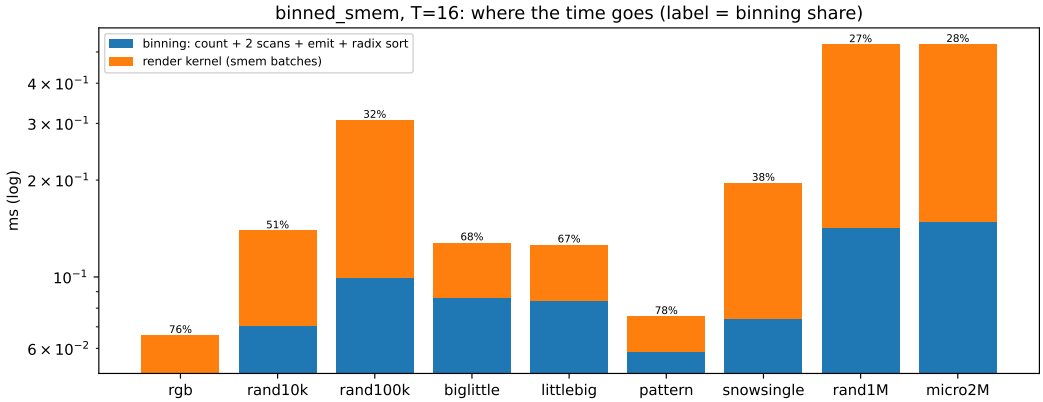


图 16: binned_smem 在 $T = 16$ 时分桶与渲染的时间分解。

图 16 中，分桶时间大致在 0.05–0.15 ms，下限由 6 次 launch、2 次 D2H 同步和一次 radix sort 决定。rgb 只有 3 个圆，分桶仍要 0.050 ms，占总时间的 82%。而且每个大圆要由单个线程把它覆盖的约 1500 个 tile 依次 `atomicAdd` 和 emit，这暴露了「一线程一圆」分桶在大圆上的负载不均衡。在 rand1M 和 micro2M 中，渲染占 72% 和 65%（0.385 和 0.380 ms），分桶的固定成本被摊薄了。

tile 大小同时影响分桶和渲染两头，下面两张图分别从时间和列表长度两个角度看这个权衡。

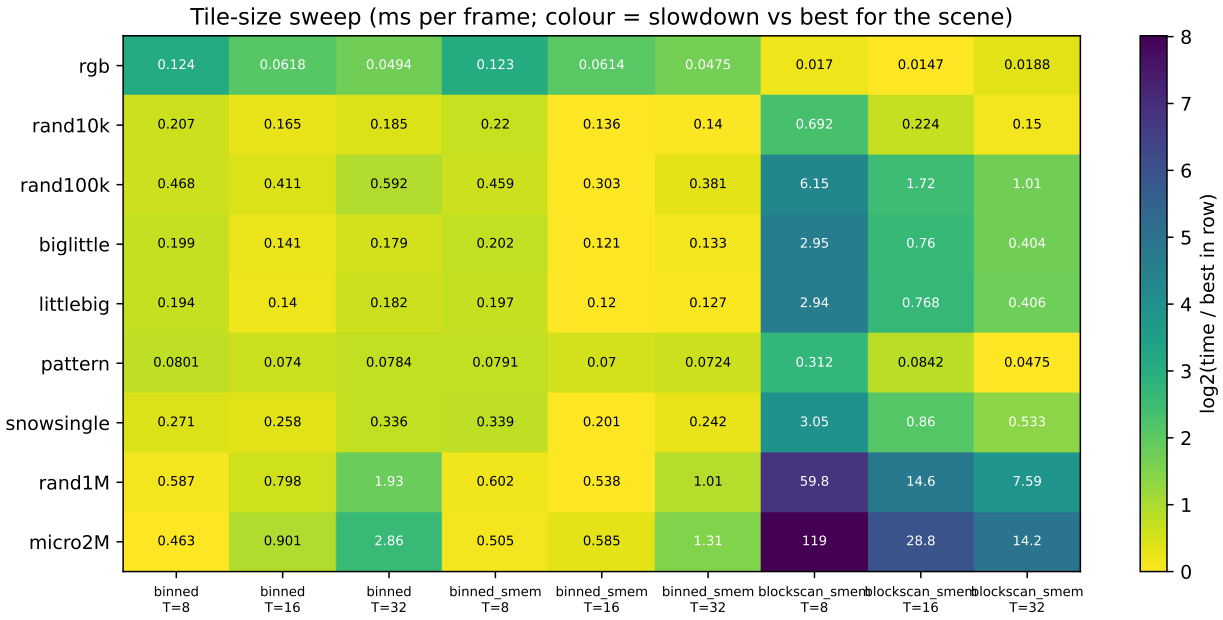


图 17: tile 大小扫描。格内为每帧 ms，颜色为相对该场景最优值的 $\log_2$ 慢速比。

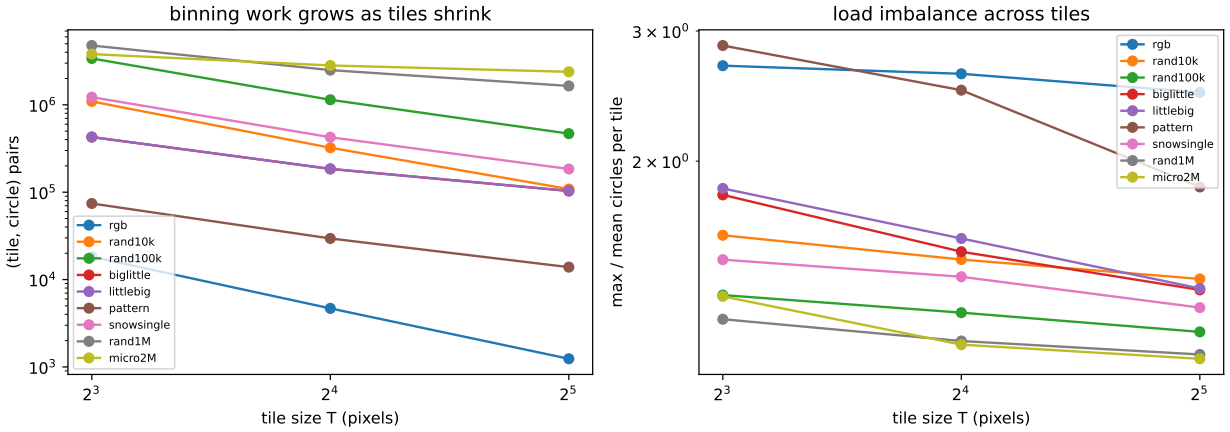


图 18: 左：(tile, circle) 对的数量；右：最长列表与平均列表长度之比（负载不均衡程度）。

图 17 and 18 说明 tile 大小是一个两头受限的权衡。tile 越小，(tile, circle) 对越多，分桶的工作量越大：rand10k 在 $T = 8$ 时有 1093947 对，$T = 32$ 时只有 108651 对。tile 越大，每个像素要遍历的列表就越长。最优点落在哪里，取决于圆的大小。micro2M 的圆半径约 1 像素，列表长度近似与 tile 面积成正比，$T = 8, 16, 32$ 时分别为 232、689、2332，所以 $T = 8$ 最快（binned 0.463 ms），$T = 32$ 慢 6.2 倍（2.858 ms）。只有 3 个大圆的 rgb 则在 $T = 32$ 时最快（binned_smem 0.047 ms），因为每个圆本来就跨很多 tile，小 tile 只会让对数变多。blockscan 的情况不同：tile 越大，tile 数越少，包围盒测试的总量（tile 数 $\times N$）就越少，所以除了 rgb（$N = 3$，测试量可以忽略），它都是 $T = 32$ 最快。负载均衡方面，rand 类场景的 max/mean 在 1.1–1.6 倍，分布相当均匀；rgb 和 pattern 里大片空白 tile 与重叠区并存，达到 1.8–2.9 倍。

tile 大小还会通过 block 大小影响资源占用，表 7 列出了各渲染 kernel 的数据。

表 7: 渲染 kernel 的资源占用（`render_attrs.jsonl`；smem 版的动态 shared memory 为 7× blockDim × 4 B）

kernel	寄存器	256 线程：blocks/SM	occupancy	1024 线程：smem (B)	occupancy
k_naive	52	4	0.67	0	0.67
k_allcircles	40	6	1.00	0	0.67
k_bin_count / k_bin_emit	22 / 40	6	1.00	0	0.67
k_render_binned	52	4	0.67	0	0.67
k_render_binned_smem	34	6	1.00	28672	0.67
k_render_blockscan	34	6	1.00	28672	0.67

$T = 32$ 对应 1024 线程的 block，每个 SM 只放得下一个，occupancy 上限是 1024/1536=0.67。相比之下，$T = 16$ 的 256 线程块每 SM 可以放 6 个，occupancy 为 1.0。所以 $T = 32$ 一开始就吃了亏。

结论：渲染器

保序的关键是把并行轴从「圆」换成「像素 / tile」，并用 scan 算偏移，生成每个 tile 内仍按输入顺序排列的圆列表；之后每个像素在寄存器里按顺序混合，不需要原子操作，也不需要锁。速度主要来自测试次数的减少（rand1M 少 1640 倍），shared memory 批量取圆再带来 1.3–2.1 倍；在本测试集中，9 个场景里有 7 个的 binned_smem 最优值落在 $T = 16$。

5 与理论模型的对照

前面的结论都来自测量。这一节反过来，用简单的模型去预测这些数字，看模型能解释多少、解释不了的又是什么。

5.1 SAXPY：屋顶线与 PCIe

SAXPY 的算术强度 $I = 2/12 = 0.167$ FLOP/B。RTX 5090 的 FP32 峰值约 105 TFLOP/s（厂商规格），屋脊点（算力受限和带宽受限的分界）约为 $105 \times 10^{12}/1.79 \times 10^{12} \approx 58$ FLOP/B，SAXPY 远在它的左边，所以 kernel 时间只取决于搬多少字节：

$$t_{\text{kernel}} = \frac{12N}{B_{\text{DRAM}}} = \frac{3.22\ \text{GB}}{1.79\ \text{TB/s}} = 1.80\ \text{ms} \quad (\text{实测}2.04\ \text{ms}，88\%).$$

端到端时间再加上两次拷贝（使用实测的单向带宽）：

$$t_{\text{e2e}} = \frac{8N}{B_{\text{H2D}}} + t_{\text{kernel}} + \frac{4N}{B_{\text{D2H}}} = \frac{2.147}{56.45} + 2.04 + \frac{1.074}{54.10}\ \text{ms} = 38.0 + 2.0 + 19.8 = 59.9\ \text{ms},$$

与实测的 59.91 ms 一致。需要说明，这几个量取自同一行数据，所以这只是自洽性检查，不是独立的预测。如果拷贝和计算完全流水化，时间下限是 $\max(8N/B_{\text{H2D}}, 4N/B_{\text{D2H}}) = 38.0$ ms。streams 实测 41.8 ms，多出的部分来自首尾两块无法重叠。

5.2 scan：访存遍数模型

既然大 N 时 scan 跑在带宽上限，最直接的模型就是「时间 = 字节数 / 带宽」。取持续显存带宽 $B = 1.52$ TB/s（cub 在 2^{28} 时的值）：

$$\begin{aligned}
\text{cub（单遍 look-back）} &: 8N\ \text{B} \Rightarrow 2.147\ \text{GB}/B = 1.41\ \text{ms} \quad (\text{实测}1.412) \\
\text{smem / raking（两遍 + 递归）} &: 16N\ \text{B} \Rightarrow 2.82\ \text{ms} \quad (\text{实测}2.817\ /\ 2.839) \\
\text{naive（逐层跨步）} &: \approx 84N\ \text{B} \Rightarrow 14.8\ \text{ms} \quad (\text{实测}16.75)
\end{aligned}$$

cub 和 smem 的字节数一目了然，naive 的 $84N$ 需要估算一下。DRAM 和 L2 的访问粒度是 32 B 的 sector（8 个 int），哪怕只用其中 4 字节，也要搬整个 sector。stride $s = 2^{d+1} < 16$ 的前三层几乎是连续访问，每层大约读写整个数组（$8N$ 字节），up-sweep 和 down-sweep 合计约 $48N$。到了 $s \geq 16$ 的层，每个线程的两个地址落在不同的 sector 上：up-sweep 读 2 个、写 1 个 sector（96 B），down-sweep 读 2 个、写 2 个（128 B）。线程数为 N/s，而 $\sum_{s\geq 16} 1/s = 1/8$，所以这部分合计 $(96+128)N/8 = 28N$。再加上初始复制的 $8N$，总计约 $84N$。实测比估算慢 13%，多出的部分包括 56 次 launch（约 0.1 ms），以及深层访问的 DRAM 行局部性较差。这个模型说明，naive 慢的主要原因是跨步访问浪费了 sector，而这一点从 work（加法次数）上完全看不出来。

work 与 span 在 GPU 上对应什么。 numpy 版统计的 span $= 2\log_2 m$，到了 naive GPU 版就是 $2\log_2 m$ 次 launch。按每次约 $2\,\mu$s 算，$N = 2^{20}$ 时的下限约为 $40 \times 2 = 80\,\mu$s，实测 83.3 μs，可见 naive 在这个规模下完全由 span（launch 次数）决定。分块版本把大部分 span 放进块内，用 `__syncthreads` 完成，每步代价是几十纳秒而不是微秒；跨块的同步只剩递归层数（$\log_{1024} N$），launch 数为 3–5 次。Brent [6] 定理 $T_P \leq W/P + S$ 在 GPU 上依然成立，只是跨块的 S 每一步要乘以约 $2\,\mu$s，而块内的每一步只是一次 barrier。

5.3 渲染器的成本模型

渲染器的三类做法，工作量结构不同。设 P 为像素数，N 为圆数，T 为 tile 边长，$\overline{L}(T)$ 为平均每 tile 的列表长度，c 为平均覆盖数，则

$$t_{\text{allcircles}} \approx \frac{P\,N}{R_{\text{test}}}, \qquad t_{\text{binned}} \approx t_{\text{bin}}(N, \#\text{pairs}) + \frac{P\,\overline{L}(T)}{R_{\text{test}}}, \qquad t_{\text{blockscan}} \approx \frac{(P/T^2)\,N}{R_{\text{bbox}}} + \frac{P\,\overline{L}(T)}{R_{\text{test}}},$$

对于半径为 r 像素的圆，$\overline{L}(T) \approx c\,(1+2r/T)^2$。从 rand1M 的数据可以得到 $R_{\text{test}} \approx 1.2\text{–}1.7\times10^{12}$/s；从 micro2M 的 blockscan（$T = 16$，8.2×10^9 次包围盒测试，28.8 ms）可以得到 $R_{\text{bbox}} \approx 2.8\times10^{11}$/s。按这个模型，blockscan 与 binned 的交点在 $N\,P/T^2 \approx R_{\text{bbox}} \cdot t_{\text{bin}}$，即 $N \approx 2.8\times10^{11} \times 10^{-4}\,\text{s}/4096 \approx 7000$（$T = 16$、$t_{\text{bin}} \approx 0.1$ ms），与 sweep 实测的约 5000 处于同一量级。模型粗糙，但足以说明交点为什么会落在几千个圆这个范围。

6 面试专题

这一节把前面的结果整理成面试里能直接说出口的话。

20 秒讲法。 「我在 RTX 5090 上复刻了 CS149 作业 3。SAXPY 的 kernel 能跑到显存带宽的 88%，但加上 PCIe 拷贝后只剩 54 GB/s，kernel 只占 3% 的时间。scan 上，我写的分块版本和 cub 都已经跑在显存带宽上限，cub 快一倍，是因为它只读写一遍。渲染器的难点是保序：一线程一圆的写法在 9 个场景里有 24% 到 100% 的像素出错。我改成按 tile 并行，用 scan 和稳定排序为每个 tile 生成按输入顺序排列的圆列表，再用 shared memory 分批取圆。结果与 CPU 参考逐位相同，rand1M 比 CPU 快 712 倍。」

STAR 故事。 情境（S）：渲染器有原子和保序两个正确性约束，而 starter 的一线程一圆 kernel 在 rand10k 上全部 1048576 个像素出错，两次运行有 591059 个像素不同。任务（T）：在逐位正确的前提下，让 rand1M、micro2M 这类上百万个圆的场景做到亚毫秒级。行动（A）：我先写了一线程一像素、遍历全部圆的版本，确认能做到逐位正确，为此加了 `-fmad=false`，并让 host 和 device 共用 `shade()`。然后加上基于 scan 的全局分桶和稳定 radix sort；接着用 shared memory 分批取圆，寄存器数从 52 降到 34；最后扫描 tile 大小，又加了一个 overdraw 固定、N 变化的 sweep，把各方

法的渐近行为分开。结果（R）：rand1M 从 844 ms 降到 0.538 ms（1569 倍），比 CPU 参考快 712 倍；90 个 (场景, 阶段, tile) 组合全部通过逐像素比较。另外还发现 blockscan 在 $N \lesssim 5000$ 时更快，tile 大小应按圆半径选择。

面试追问：为什么一线程一圆、每像素加 atomic 也不对？

因为 atomic 只保证一次「读–改–写」不被打断，不保证不同圆之间的先后顺序，而 over 混合（式 (1)）不满足交换律，两个 $\alpha = 0.5$ 的圆换个顺序，像素颜色就不同。何况 float4 颜色没有现成的原子 over 操作，只能用 CAS 循环或锁来实现，而且依然无法保序。正确的做法是按像素并行，每个像素按顺序遍历覆盖它的圆。

常见的错答是「加 `atomicAdd` 就好了」。加法满足交换律，over 不满足；rgb 场景即使没有竞争（结果可复现），也有 24.3% 的像素出错。

面试追问：SAXPY 放到 GPU 上为什么没变快？

因为它的算术强度只有 0.17 FLOP/B，kernel 受显存带宽限制，1578 GB/s 已经是 88% 的峰值；而端到端还要过 PCIe，pinned 内存只有 53.8 GB/s，比显存慢 29 倍，kernel 只占 3.4% 的时间。pageable 内存还要多一次 CPU staging 拷贝，只有 10.5 GB/s。要让 GPU 有收益，数据要常驻显存，或者用 stream 让拷贝与计算重叠，或者把多个操作融合起来，提高每字节传输对应的计算量。

常见的错答有三种：「GPU 算力强，所以一定更快」；只报 kernel 时间、忽略拷贝；以为 PCIe 是半双工，而实际上 streams 版本达到了 77 GB/s。

面试追问：Blelloch 的 work 是 $O(N)$，为什么 naive GPU 版仍比 cub 慢 12 倍？

因为 GPU 上的成本是字节数和同步次数，而不是加法次数。naive 的 launch 数为 $2\log_2 m$（2^{28} 时 56 次）；更要紧的是深层的跨步访问，每个线程只用 4 字节却要搬 32 字节的 sector，总流量约 $84N$ 字节，而 cub 只要 $8N$。

常见的错答是「Blelloch 是 work-efficient 的，所以在 GPU 上已经最优」。work-efficient 只说明加法少，不说明访存少。

面试追问：cub 的单遍 scan 是怎么做到只读写一遍的？

靠 decoupled look-back[5]。每个 tile 扫完自己的数据后，先发布本块的局部和（状态 A），再往前看：遇到已经发布全局前缀的 tile（状态 P）就停下，否则累加前面各块的局部和。这样块间依赖通过 L2 里的状态字解决，既不需要第二次 launch，也不需要把数据再读一遍。实测 2^{28} 时 1.41 ms，与 $8N/1.52$ TB/s 的预测一致。

常见的错答是「每个块等前一个块算完再开始」。那是一条串行链，look-back 正是为了避免这种等待。

面试追问：为什么 bank-conflict padding 有用？代价是什么？

因为 up-sweep 第 d 层的访问步长是 2^{d+1}，步长为 32 的倍数时，一个 warp 的访问全落在同一个 bank 上，只能串行。每 32 个元素插 1 个空位（`i + (i>>5)`），就能把这些访问错开到不同的 bank。实测 2^{24} 时从 146.9 降到 112.1 μs。代价是每块多用 1/32 的 shared memory（1024 个元素多 32 个 int），以及每次访问多一次移位和加法。

常见的错答是「padding 是为了对齐」。它的目的是打乱 bank 映射，与对齐无关。

面试追问：为什么不对 (tile, circle) 对做全局排序也能保序？你的实现里为什么又用了 radix sort？

保序只要求每个 tile 内的圆按下标递增排列。按圆的下标做 exclusive scan 得到偏移，emit 出来的数组本来就按圆排好了，之后只需要一次稳定的按 tile 分组。分组可以用稳定的 radix sort，只排 $\lceil \log_2 \#\text{tiles} \rceil$ 位；也可以在 tile 内按圆分块、对命中标志做 scan 压缩（blockscan 就是这样），完全不用排序。无论哪种，稳定性都不能丢：用不稳定的排序，同一 tile 内的顺序会被打乱，图像就会出错。

常见的错答是「按 (tile, depth) 排序」。CS149 的场景没有给出可比较的深度值，顺序就是输入下标。

面试追问：tile 大小怎么选？

按圆的半径选：圆小用小 tile，圆大用大 tile。(tile, circle) 对的数量约为 $N(1+2r/T)^2$，tile 越小越多；每像素的列表长度约为 $c(1+2r/T)^2$，覆盖数 c 固定时，tile 越大，包围盒被放大的部分越多，列表就越长。半径约 1 像素的 micro2M 在 $T=8$ 时最快，$T=32$ 慢 6.2 倍；大圆场景则是 $T=32$ 最快。另外，$T=32$ 对应 1024 线程的 block，每个 SM 只能放一个，occupancy 上限为 0.67。在本测试集中，$T=16$ 是最稳妥的默认值。

常见的错答是「tile 越小，并行度越高，就越快」。tile 越小，分桶的对数和排序开销都会增加。

面试追问：小数组的 scan 为什么不该放在 GPU 上？

因为 GPU 的单次调用延迟太高。每次 launch 约 2 μs，加上同步约 6 μs；$N=1000$ 时最快的 GPU 版本也要 4.4 μs，而 CPU 顺序扫描 1000 个 int 只需几百纳秒。GPU 的优势在吞吐，这类小工作量应该留在 CPU 上做，或者合并进一个大 kernel（kernel fusion、persistent kernel）。

常见的错答是「thrust 很快，可以随便调用」。thrust 每次调用都要分配临时缓冲区，$N=1000$ 时用了 12.3 μs，是最慢的之一。

面试追问：find_repeats 为什么比 thrust::copy_if 慢 6.6 倍？怎么改？

因为遍数多。我的实现分三遍（flags、scan、scatter），scan 本身又是两遍，总流量约 $32N$ 字节，按 1.87 ms 算已经是 1.71 TB/s，贴近带宽上限；copy_if 把谓词计算、scan 和压缩合在一个 kernel 里，只读一遍 a。改进方法是融合：在 scan 的 tile 内部直接计算谓词，look-back 拿到块前缀之后立刻 scatter。

常见的错答是「scan 部分写得不够快」。实际上每个阶段都已经接近带宽上限，问题出在遍数。

面试追问：这个渲染器和 3D Gaussian Splatting 的光栅化有什么关系？

思路是同一套 [10]：每个 Gaussian 先算出覆盖哪些 16×16 tile，生成 (tile|depth, id) 键值对，用 radix sort 排序，再让每个 tile 用一个 block 按顺序做 α 混合，并用 shared memory 分批取数据。区别在于 3DGS 按深度而不是输入下标排序，并且累计透明度足够低时会提前终止。本实验里的 tile 大小权衡和分桶开销占比，在 3DGS 中同样存在。

常见的错答是「3DGS 用的是硬件光栅化」。它用的是 CUDA 实现的 tile 软件光栅化，与本实验的 binned_smem 属于同一类方法。

7 局限

下面这些限制决定了哪些结论可以外推、哪些不能。

不能推出的结论

- 没有 ncu 数据。容器没有性能计数器权限（`ERR_NVGPUCTRPERM`），所以没有实测的 achieved occupancy、DRAM 吞吐和 bank conflict 次数。文中的 occupancy 是 API 给出的理论值，DRAM 流量是根据访存模式推算的字节数，padding 消除 bank conflict 的结论只有时间上的证据。
- 不能推出「GPU 渲染比 CPU 快 712 倍」具有普遍性。CPU 参考是单线程、未向量化的 C++；换成 16 核加 SIMD 的 CPU 实现，差距会缩小一到两个数量级。
- 场景只是「类似官方」，不是官方场景。官方 starter 的场景文件和雪花颜色表无法离线获得，本文的场景参数是自己定义的（表 3），时间不能直接与课程排行榜比较。
- snow 场景不是逐位相等。误差为 2.98×10^{-7}，来自 device 端 `expf` 与 glibc 实现的差异；其余场景依赖 `-fmad=false`，打开 FMA 后会有 ULP 级的差别。
- L2 效应。$N = 2^{20}$–2^{24} 的带宽数字（如 SAXPY 的 4759 GB/s）是 L2 带宽，不能当作显存带宽引用。
- 分桶时间包含 D2H 同步。读取 pairs 总数需要两次 4 字节 D2H，这是本实现的选择；预分配上界或使用 CUDA graph 可以去掉它，所以小场景的分桶时间（约 0.05 ms）偏高。
- 只有一块 GPU、一次运行。每个点是 5–50 次重复的中位数，没有报告方差；与其他用户共享机器时，GPU 锁保证了独占，但 CPU 和 PCIe 的状态无法完全控制。

8 结论

回到开头的三个问题，读者可以带走这几点：

1. SAXPY 的 kernel 跑到 1578 GB/s（88% 峰值），端到端却只有 pinned 53.8、pageable 10.5、streams 77.1 GB/s，kernel 只占端到端时间的 3.4%。访存受限的 kernel，瓶颈在 PCIe。
2. 六种 scan 在 19 个规模（包括非 2 的幂）上全部正确。大规模下的速度由访存遍数决定（$84N$、$16N$、$8N$ 字节），三类实现折算后都在约 1.52 TB/s；小规模下由 launch 数决定，每次约 2 μs。naive 补齐到 2 的幂的代价在 10^8 时为 1.37 倍。
3. 自写的三遍 `find_repeats` 在 10^8 时用 1.87 ms，比单遍的 thrust::copy_if 慢 6.6 倍。差距来自访存遍数，可以靠 kernel 融合消除。
4. 渲染器方面，starter 式 kernel 在除 rgb 外的 8 个场景中有 48.5%–100% 的像素出错（rgb 为 24.3%），且多数场景结果不可复现；四个正确阶段全部与 CPU 参考逐位一致。分桶让测试次数减少约 1640 倍，shared memory 批量取圆再带来 1.3–2.1 倍。$T = 16$ 是稳妥的默认值，圆很小时用 $T = 8$；$N \lesssim 5000$ 时，单次 launch 的 blockscan 更快。

参考文献

[1] G. E. Blelloch. Prefix sums and their applications. Technical Report CMU-CS-90-190, Carnegie Mellon University, 1990.

[2] W. D. Hillis and G. L. Steele Jr. Data parallel algorithms. *Communications of the ACM*, 29(12):1170–1183, 1986.

[3] M. Harris, S. Sengupta, and J. D. Owens. Parallel prefix sum (scan) with CUDA. In *GPU Gems 3*, chapter 39. Addison-Wesley, 2007.

[4] S. Sengupta, M. Harris, Y. Zhang, and J. D. Owens. Scan primitives for GPU computing. In *Graphics Hardware*, 2007.

[5] D. Merrill and M. Garland. Single-pass parallel prefix scan with decoupled look-back. NVIDIA Technical Report NVR-2016-002, 2016.

[6] R. P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, 1974.

[7] T. Porter and T. Duff. Compositing digital images. In *SIGGRAPH*, 1984.

[8] S. Laine and T. Karras. High-performance software rasterization on GPUs. In *High Performance Graphics*, 2011.

[9] S. Williams, A. Waterman, and D. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[10] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

[11] Stanford CS149 Fall 2024. Assignment 3: A simple renderer in CUDA. `https://github.com/stanford-cs149/asst3`.